MOCHILA 0/1

PROGRAMAÇÃO DINÂMICA





GRUPO

LEONARDO HENRIQUE CARVALHO OLIVEIRA - 12311BCC003

RAQUEL EMILLENE FREIRE THOMÉ - 12311BCC026

YAN LUCAS SANTOS MARRA - 12311BCC013

SUMÁRIO

01 Descrição do problema

02 Prova da subestrutura ótima

03 Recorrência

04 Algoritmos

05 Exemplo

06 Análise

07 Gráficos

08 Referências

DEFINIÇÃO

Contexto:

- Um ladrão assalta uma loja e encontra n itens.
- Cada item possui um peso e um valor

Restrições:

- A mochila suporta no máximo W quilos

Objetivo:

- Escolher os itens de forma a obter a carga mais valiosa possível.

Condições:

- Para cada item
  - Levar (1) ou não levar (0).
  - Não é permitido pegar apenas uma fração do item.
  - Não é permitido levar mais de uma vez o mesmo item.



PROVA SUBESTRUTURA

Seja $I(n, W)$ o problema da mochila com n itens e capacidade W.

Seja S uma solução ótima de $I(n, W)$, onde S $\subseteq$ {1, ..., n}.

Ao analisar o último elemento do conjunto {1, ..., n}, conseguimos decompor o problema original em subproblemas menores. Isso porque qualquer solução ótima deve se enquadrar em um dos dois casos possíveis:

1. Item n pertence à solução ótima, ou
2. Item n não pertence à solução ótima.

Assim, conseguimos dividir todas as soluções possíveis nesses dois cenários.

PROVA SUBESTRUTURA

Caso 1: $n \notin S$

Se n não está em S, então S deve ser uma solução ótima para a entrada I(n-1, W).

Por contradição, suponha que exista S*, solução ótima para I(n-1, W), com v(S*) > v(S).

Como: $\sum i \in S^* \; pi \leq W$ e $S^* \subseteq \{1, ..., n\text{-}1\} \subseteq \{1, ..., n\}$, então S* também seria uma solução viável para I (n, W).

Isso contradiz a hipótese de que S é solução ótima para I(n, W).

Caso 2: $n \in$ S

Se n pertence a S, defina $S' = S\text{-}\{n\}$, $S' \subseteq \{1, ..., n\text{-}1\}$. O peso total de S' é: $\sum i \in S' \; pi = \sum i \in S \setminus \{n\} \; pi = \sum i \in S \; pi - pn \leq W - pn$.

Logo, S' é solução viável de I(n-1, W-pn).

Por contradição, suponha que exista S* solução ótima de I(n-1, W-pn), com $v\,(S^*) > v(S')$.

Se adicionarmos o item n a S*, obtemos: $v\,(S^* \cup \{n\}) = v(S^*) + vn > v(S') + vn = v(S)$. Isso contradiz a hipótese de que S é ótima para I(n, W).

CONCLUSÃO

Portanto, em ambos os casos, mostramos que a solução ótima S para I(n, W) é formada por uma solução ótima de um subproblema menor. Isso prova que o problema da mochila 0/1 possui subestrutura ótima.



RECORRÊNCIA

Definindo M(i, W) como o valor máximo que podemos obter escolhendo itens apenas do conjunto {1, ..., i} quando a capacidade disponível é W.

Casos-base:

Se i = 0 (nenhum item disponível) ou w=0 (capacidade zero), não há valor a obter: M(i, w) = 0.

Ideia central (subestrutura ótima)

Para cada item i e capacidade w, a solução ótima depende apenas de subproblemas menores:

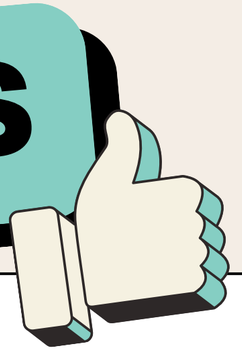
- Não incluir i: o valor ótimo é M(i-1, w).
- Incluir i (se couber): o valor ótimo é vi + M(i-1, w-pi).

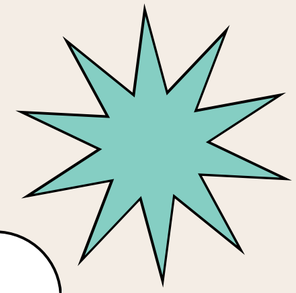
Assim, a recorrência surge naturalmente como o máximo entre essas duas opções.

RECORRÊNCIA

$$M(i,w) = \begin{cases} 0, & \text{se } i = 0 \text{ ou } w = 0 \\ M(i-1,w), & \text{se } p_i > w \\ \max\left(M(i-1,w),\ v_i + M(i-1,w-p_i)\right), & \text{se } p_i \leq w \end{cases}$$

ALGORITMOS



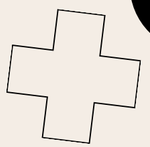


```
Função mochila(n, W):
    Se n == 0 ou W == 0:
        Retornar 0

    excluir = mochila(n-1, W)

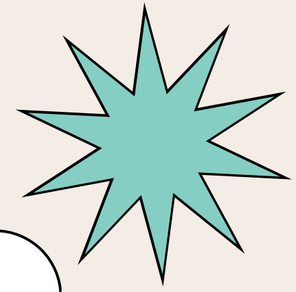
    incluir = 0
    Se W >= peso[n]:
        incluir = mochila(n-1, W - peso[n]) + valor[n]

    Retornar max(incluir, excluir)
```



ALGORITMOS

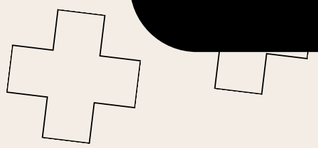




```
Função Mochila0_1(n, W, peso[1..n], valor[1..n])
    dp[0..n][0..W]

    Para i de 0 até n
        Para w de 0 até W
            Se i == 0 ou w == 0
                dp[i][w] = 0
            Senao se peso[i] <= w
                dp[i][w] = max(dp[i-1][w], valor[i] + dp[i-1][w - peso[i]])
            Senao
                dp[i][w] = dp[i-1][w]

    retorne dp[n][W]
```



ALGORITMOS

```
inicializar itens_escolhidos como lista vazia
i ← n
j ← C

enquanto i > 0 e j > 0 faça
    se matriz[i][j] ≠ matriz[i-1][j] então
        adicionar i à lista itens_escolhidos
        j ← j - pesos[i-1]
    fim se
    i ← i - 1
fim enquanto

retornar itens_escolhidos
```

Pseucódigo da construção da solução ótima

Dados do Problema:

Capacidade da Mochila: C = 10

Itens Disponíveis:

- Item 1: (valor=3, peso=2)
- Item 2: (valor=4, peso=3)
- Item 3: (valor=8, peso=4)
- Item 4: (valor=8, peso=5)
- Item 5: (valor=10, peso=9)

Matriz de Programação Dinâmica

dp[i][w] = valor máximo usando itens 0...i-1 com capacidade w

i\w	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7	7
3	0	0	3	4	8	8	11	12	12	15	15
4	0	0	3	4	8	8	11	12	12	16	16
5	0	0	3	4	8	8	11	12	12	16	16

Localização do custo da solução ótima: dp:[5][10]



RESULTADOS DO EXEMPLO ANTERIOR

```
============================================================
 Teste com 5 itens | Capacidade: 10
============================================================
                  mochila -> Resultado:  16 | Tempo: 0.000000s
               mochila_PD -> Resultado:  16 | Tempo: 0.000000s
  mochila_PD_com_solucao -> Resultado:  16 | Itens: [3, 4] | Tempo: 0.000000s
```

Custo da solução ótima

Solução ótima



ANÁLISE DA RECURSÃO

Análise de Tempo:

O tempo de execução, $T(n, W)$, pode ser definido pela seguinte relação de **recorrência**, que reflete as ramificações de decisão do algoritmo:

$T(n, W) = \Theta(1)$, se $n = 0$ ou $W = 0$

$T(n, W) = T(n - 1, W) + T(n - 1, W - wn) + \Theta(1)$, se $n > 0$

Simplificando

$M(n) = \Theta(1)$, se $n = 0$

$M(n) = 2M(n - 1) + \Theta(1)$, se $n > 0$

Aplicando o método da expansão:

1º - $M(n) = 2M(n - 1) + 1$

2º - $M(n) = 4M(n - 2) + 2 + 1$

3º - $M(n) = 8M(n - 3) + 4 + 2 + 1$

4º - $M(n) = 16M(n - 4) + 8 + 4 + 2 + 1$

Após i iterações:

$$M(n) = 2^i * M(n - i) + \sum_{j=0}^{i-1} 2^j$$

$$\mathbf{(2 \cdot 2^n) - 1 = \Theta(2^n)}$$

ANÁLISE DA RECURSÃO

Análise de Espaço:

A complexidade de espaço de um algoritmo recursivo é determinada pela profundidade máxima de sua **pilha de chamadas**.

No pior caso da Mochila 0/1, a recursão pode descer até a profundidade n, pois a cada passo é feita uma chamada para o subproblema com n-1 itens, acumulando chamadas sem resolver as anteriores até atingir o caso base.

Profundidade é proporcional a quantidade de itens n.
Logo a complexidade de espaço é igual a **Θ(n)**

ANÁLISE DO ITERATIVO

- O problema possui a propriedade de subestrutura ótima, pois **a solução de um problema maior depende das soluções ótimas de subproblemas menores.**
- Cada subproblema pode ser definido por um par (i, w), onde $0 \leq i \leq N$ e $0 \leq w \leq W$.
- O número total de subproblemas é a quantidade de combinações possíveis desses pares, o que resulta em um número de subproblemas na ordem de $\Theta(N \cdot W)$.

Complexidade de Tempo:

- Dois laços aninhados, um varrendo os itens até N e outro varrendo a capacidade até W
- O tempo é proporcional ao número de iterações, isto é, complexida de tempo igual a $\Theta(N \cdot W)$

Complexidade de Espaço:

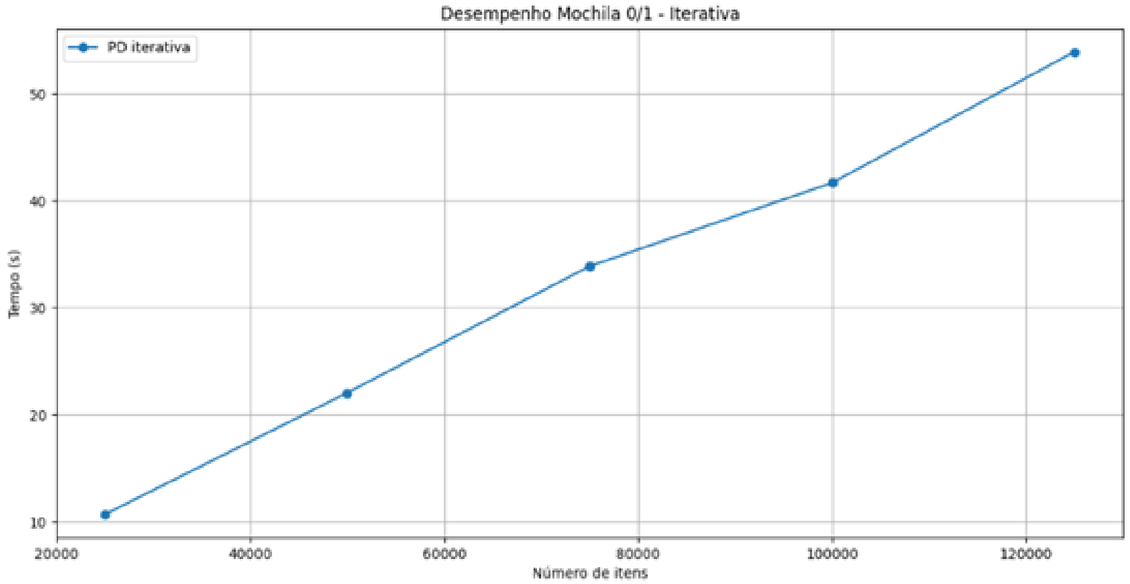
- O algoritmo utiliza uma matriz de dimensões: N+1 e W+1 (Número de itens e capacidade)
- O espaço necessário é proporcional ao tamanho da matriz, portanto a complexidade de espaço também é $\Theta(N \cdot W)$

ESTATÍSTICAS

Desempenho do Algoritmo Iterativo em Python

Algoritmo Iterativo	
Qtde de itens	Tempo(seg)
20000	8.5
40000	16.9
60000	27.2
80000	37.4
100000	41.6
120000	53.8

OBS.: Consideramos uma capacidade da mochila = 1000, com os valores dos itens variando de 1 até 200 e o peso podendo variar de 1 até 100.



ESTATÍSTICAS

Desempenho do Algoritmo Iterativo em C

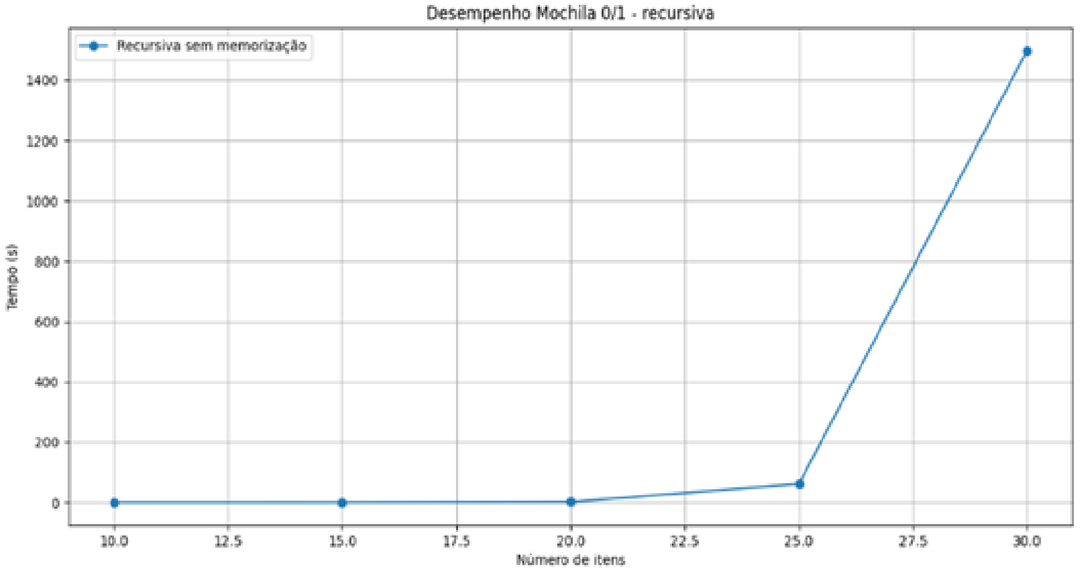
Algoritmo Iterativo em C	
Qtde de itens	Tempo(seg)
20000	0.319
40000	0.350
60000	0.648
80000	1.018
100000	1.839
120000	2.145
150000	2.303
200000	2.956
400000	3.325



ESTATÍSTICAS

Desempenho do Algoritmo recursivo em Python

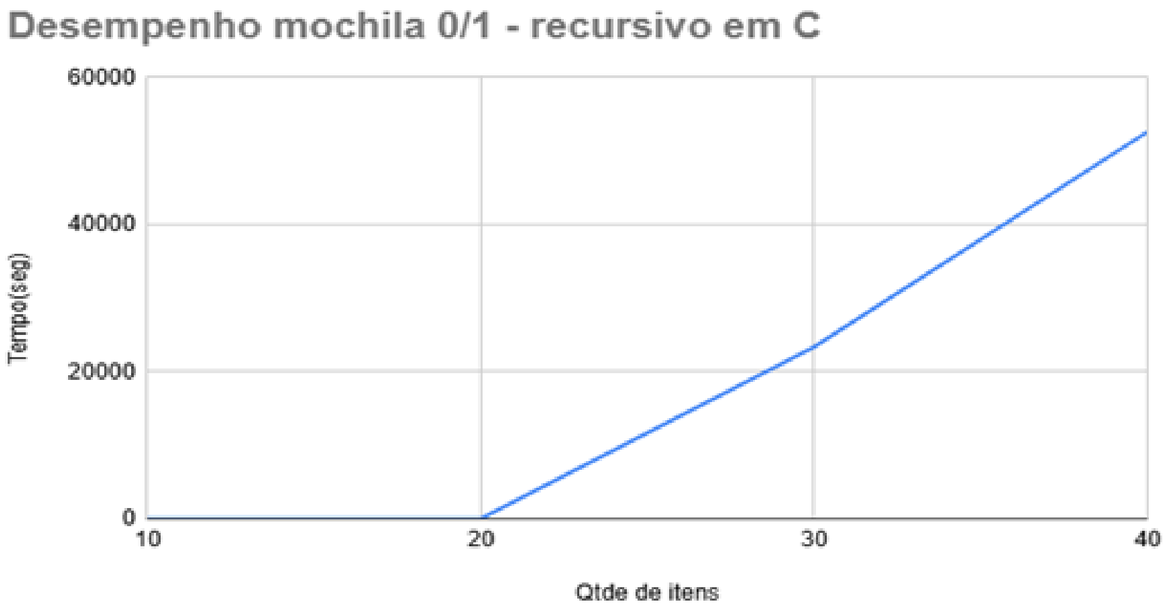
Algoritmo recursivo	
Qtde de itens	**Tempo(seg)**
10	3.005
15	7.256
20	16.101
25	65.007
30	1,505.793



ESTATÍSTICAS

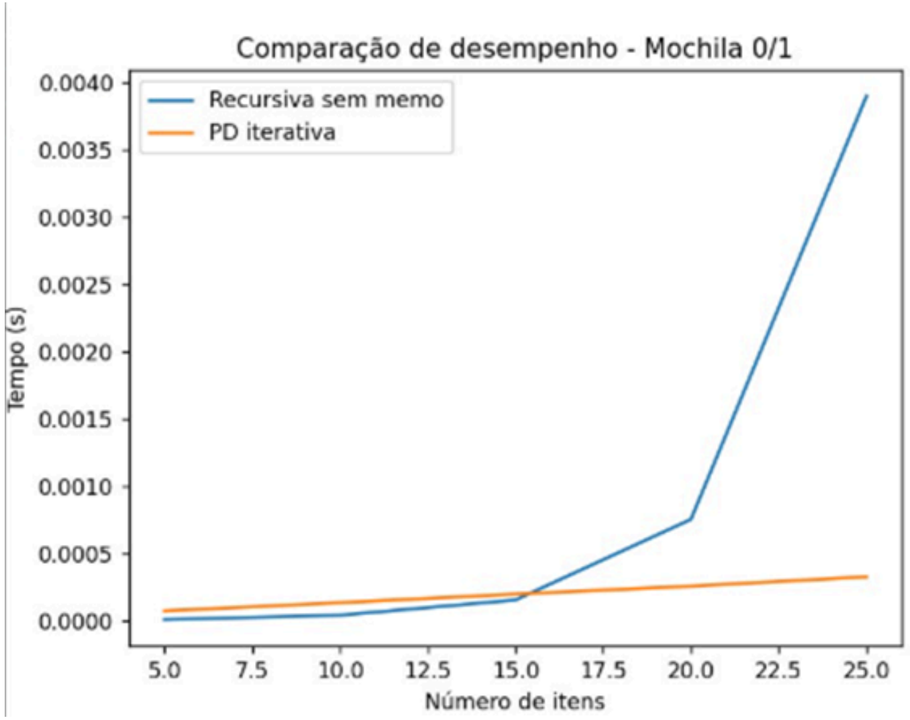
Desempenho do Algoritmo recursivo em C

Algoritmo recursivo em C	
Qtde de itens	Tempo(seg)
10	0.003
20	0.004
30	23.343
40	52.588



ESTATÍSTICAS

Comparação do método iterativo e do método recursivo



Capacidade = 100

REFERÊNCIAS

1. CORMEN, T. H. Algoritmos, Teoria e Prática. 3 ed.
2. Notas de aula do Prof. Mário César San Felice (**http://www.aloc.ufscar.br/felice**) do Departamento de Computação (DC) da Universidade Federal de São Carlos (UFSCar).
3. Notas de aula do Prof. Tim Roughgarden (http://timroughgarden.org/) do Departamento de Ciência da Computação na Columbia University.
4. Notas de aula da Prof. Carla Negri Lintzmayer (http://professor.ufabc.edu.br/~carla.negri/) do Centro de Matemática, Computação e Cognição da Universidade Federal do ABC


OBRIGADA